Solution: Fibonacci Series Using Memoization

Problem Statement

Print Fibonacci Series Using Memoization and Recursion.

Given a positive integer `n`, print the Fibonacci series up to the `n`th term using memoization and recursion.

Examples:

Sr#	Input	Output	Explanation
1	5	0, 1, 1, 2, 3	The Fibonacci series up to the 5th term is 0, 1, 1, 2, 3.
2	8	0, 1, 1, 2, 3, 5, 8, 13	The Fibonacci series up to the 8th term is 0, 1, 1, 2, 3, 5, 8, 13.
3	12	0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55	The Fibonacci series up to the 12th term is 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55.

Solution:

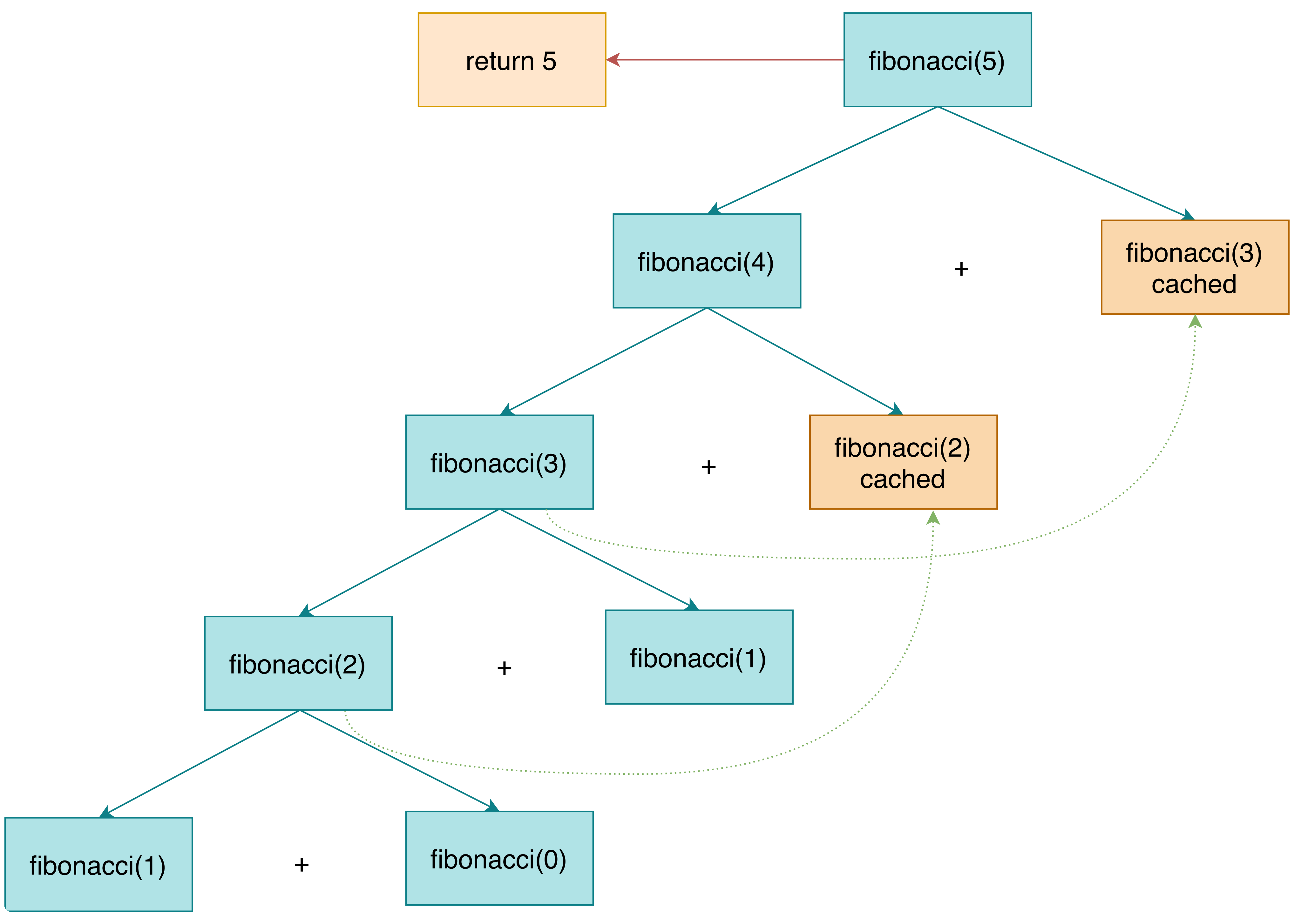
The Fibonacci series can be calculated using memoization and recursion. The key idea is to store the previously calculated Fibonacci numbers in a memoization table to avoid redundant calculations. We define a recursive function `fibonacci` that takes an input `n` and returns the Fibonacci number at position `n`.

The base case occurs when `n` is 0 or 1, in which case we return `n` as the Fibonacci number. For any other value of `n`, we check if the Fibonacci number at position `n` is already calculated and stored in the memoization table. If it is, we directly return the value. Otherwise, we recursively calculate the Fibonacci number by summing the previous two Fibonacci numbers (calculated recursively) and store it in the memoization table for future use.

The overall algorithm follows these steps:

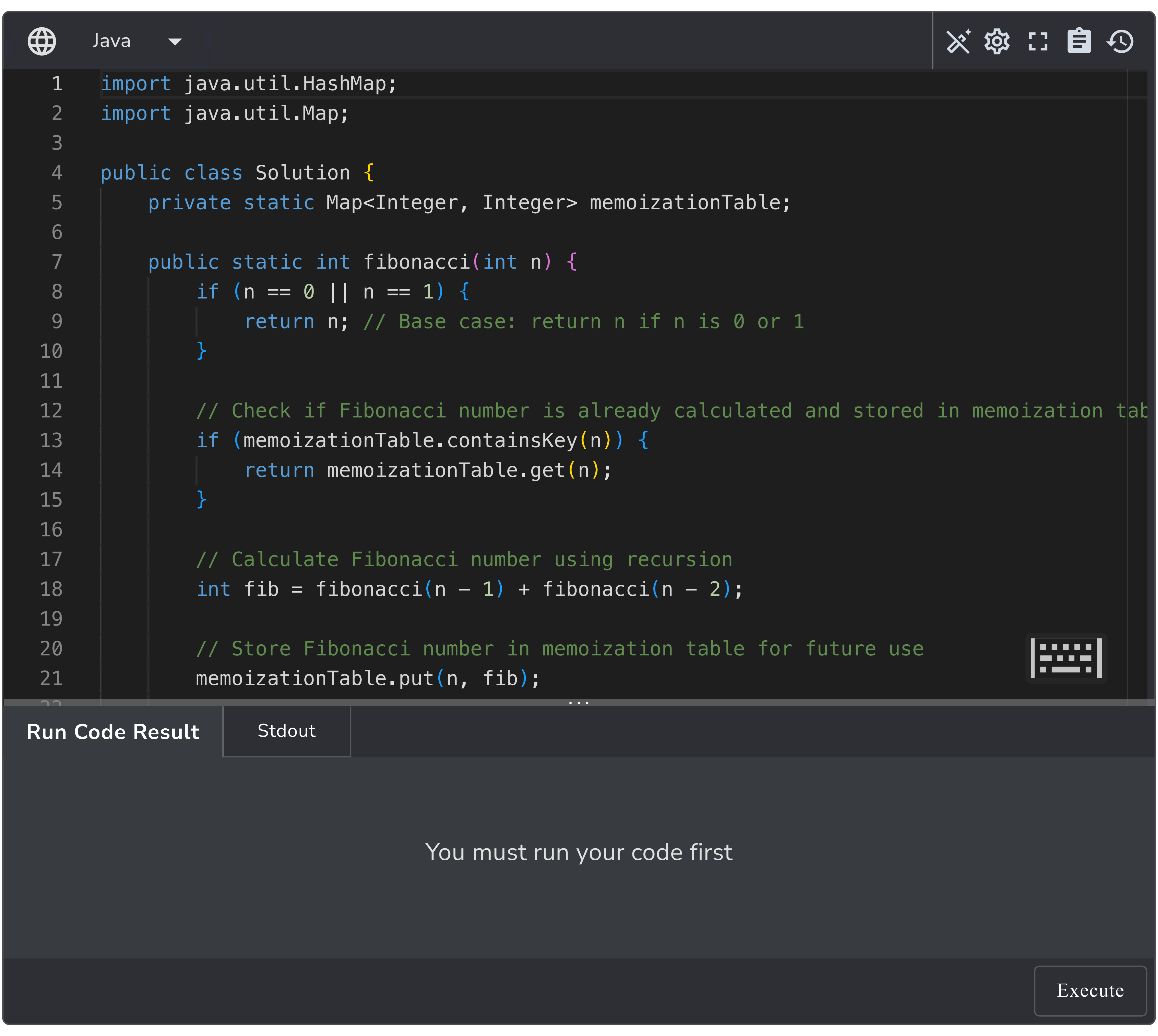
1. Initialize a memoization table to store the Fibonacci numbers.
2. Implement the recursive function `fibonacci` that takes `n`

   as input.
   - Check if `n` is 0 or 1. If so, return `n`.
   - Check if the Fibonacci number at position `n` is already present in the memoization table. If so, return the value.
   - Otherwise, calculate the Fibonacci number by recursively calling the `fibonacci` function for `n-1` and `n-2`, summing the results, and store it in the memoization table.
3. In the main function, take input `n` from the user.
4. Call the `fibonacci` function with `n` as input and print the Fibonacci series up to the `n`th term.



Code

Here is the code for this algorithm:



```java
import java.util.HashMap;
import java.util.Map;

public class Solution {
    private static Map<Integer, Integer> memoizationTable;

    public static int fibonacci(int n) {
        if (n == 0 || n == 1) {
            return n; // Base case: return n if n is 0 or 1
        }

        // Check if Fibonacci number is already calculated and stored in memoization tab
        if (memoizationTable.containsKey(n)) {
            return memoizationTable.get(n);
        }

        // Calculate Fibonacci number using recursion
        int fib = fibonacci(n - 1) + fibonacci(n - 2);

        // Store Fibonacci number in memoization table for future use
        memoizationTable.put(n, fib);
```

You must run your code first

Time and Space Complexity

The time complexity of the Fibonacci series algorithm using memoization and recursion is $O(N)$, where n is the input number. This is because we calculate the Fibonacci numbers from 0 to n, and each Fibonacci number is calculated only once and stored in the memoization table for future use.

The space complexity is also $O(N)$ because we store the Fibonacci numbers in the memoization table, which can hold up to n entries. Additionally, the recursion stack occupies space proportional to the input number n.

← Previous
16. Fibonacci Series Using Memoization
Next →
17. Good Number
☐ Mark as Completed